

Decisiones de diseno de las pruebas

Task Manager Testing Lab | Suite Jest | Documento breve (max. 2 paginas)

1. Acceso al repositorio

Codigo fuente y pruebas: <https://github.com/GayatriM916/task-manager> | Carpeta de pruebas: `__tests__` / | Cobertura: `npx jest --coverage`

2. Que se probo en esta iteracion

Se reforzaron las pruebas unitarias de utilidades y se completo el aislamiento del hook que interactua con la capa de servicio. No se modifico la logica funcional de la aplicacion ni se eliminaron pruebas existentes.

- Utils - `filterTasksByStatus` (`src/Utils/filterTasks.ts`): filtrado por estado, lista vacia, estados invalidos/nulos y uso de matchers `toEqual`, `toContain` y `toThrow`.
- Utils - `validateTaskTitle` (`src/Utils/validateTask.ts`): limites `null/undefined` y comprobacion parcial del mensaje de error con `toContain`.
- Hooks - `useCreateTask` (`src/hooks/useCreateTask.ts`): estado inicial `idle`, submit exitoso (`success + lista`), fallo del servicio (`loading -> error`) y `removeTask`.

3. Por que se eligieron esos casos

- Casos principales: validan el comportamiento esperado del dominio (filtrar `pending/completed`, titulo valido, crear tarea con exito).
- Casos limite: listas vacias, estados inesperados (`" / null`), titulos `null/undefined` y errores de servicio. Exponen reglas de validacion y manejo de fallos.
- Hooks con `renderHook + act`: permiten verificar actualizaciones de estado y efectos secundarios (llamada a `createTask`) sin montar pantallas completas.
- Criterio de suficiencia: al menos 6 pruebas en utils (sumando las nuevas) y al menos 4 pruebas nuevas para el hook sin cobertura previa.

4. Dependencias aisladas con mocking

Solo se mockeo donde habia dependencia externa. Las utilidades son funciones puras: no requieren `jest.mock` ni `jest.fn`.

- `jest.mock('../src/services/taskService')` en `__tests__/hooks/useCreateTask.test.ts`: aisla `createTask` (capa de servicio). Motivo: controlar exito/fallo sin depender de la implementacion real (`red` o `Date.now`), observar `loading/error/success` y verificar que `submit` invoca el servicio con el titulo esperado.
- `mockResolvedValue / mockImplementation + jest.clearAllMocks`: configuran respuestas por caso y evitan fugas de estado entre pruebas. No se anadieron mocks innecesarios en `filterTasks` ni `validateTask`.

5. Resultado de la suite

Ejecucion verificada con `npx jest --coverage`: 13 suites / 53 pruebas en verde. Cobertura global aproximada: 90.9% statements. Hooks y utils al 100% en statements de los archivos cubiertos por estas pruebas.